

DS n° 1**LE 18/10/24**

Il est possible dans un exercice d'utiliser les fonctions définies dans les questions précédentes. Hormis celles proposées, on n'introduira pas de bibliothèque supplémentaire.

EXERCICE 1 : (DÉ)CONSTRUCTIONS DE LISTES

On considère la fonction suivante :

```
1 def separation(L):
2     tA,tB = [],[]
3     for val in L :
4         if val %2 == 0:
5             tA.append(val)
6         else:
7             tB.append(val)
8     return tA,tB
```

L'appel se fait avec une liste L d'entiers.

- 1) Parcourir pas à pas cette fonction avec $L=[3,2,5,1,8]$.
- 2) Recopier et commenter cette fonction pour décrire son fonctionnement.
- 3) Écrire une fonction `signe(L)` qui prend en argument une liste L d'entiers et retourne deux listes, une constituée des entiers positifs et l'autre des entiers négatifs de L.
Par exemple `signe([3,-1,-2,7,5])` retourne $[3,7,5], [-1,-2]$.

EXERCICE 2 : RECHERCHES DANS UN TABLEAU**Partie A - Recherches séquentielles**

On considère un tableau `tab` d'entiers.

- 1) Proposer une fonction `recherche(tab,x)` renvoyant `True` ou `False` selon que l'entier `x` soit dans `tab` où non, via une recherche séquentielle.
- 2) Si `tab` est de taille n , quel nombre de comparaisons au pire est faite?
- 3) Proposer une fonction `recherchepos(tab,x)` qui donne la première position de `x` dans `tab`, si `x` n'est pas présent, on renverra `None`.
- 4) Proposer une fonction `ttepos(tab,x)` qui donne la liste des position de `x` dans `tab`, si `x` n'est pas présent, on renverra une liste vide.
- 5) Proposer enfin une fonction `compte(tab,x)` qui donne le nombre de fois où `x` est présent dans `tab`.

Partie B - Égalité de tableaux

On considère deux tableaux $T1$ et $T2$. En Python la commande $T1==T2$ renvoie `True` ou `False` selon que les tableaux soit identiques où non.

Par exemple si $T1 = [1,3,8]$ et $T2 = [1,3,8]$, $T1 == T2$ donne `True`, et si $T1 = [1,3,8]$ et $T2 = [2,3,8]$, $T1 == T2$ donne `False`.

- 6) Écrire une fonction `egal(tab1,tab2)` qui prends en argument deux tableaux (de même taille), et qui donne le retour de `tab1 == tab2` mais sans utiliser `==`.

On souhaite maintenant faire évoluer ce principe sur une égalité case par case avec une fonction `tabegal` Par exemple si $T1 = [1,3,8]$ et $T2 = [2,3,8]$, on l'appel `tabegal(T1,T2)` retourne `[False, True, True]`.

- 7) Écrire cette fonction `tabegal(tab1,tab2)`.

On s'intéresse maintenant aux valeurs communes de deux tableaux.

- 8) Écrire une fonction `nbegal(tab1,tab2)` qui retourne le nombre de valeurs communes des deux tableaux, supposé de même taille, aux même positions. Par exemple si $T1 = [1,3,8]$ et $T2 = [8,3,5]$, on l'appel `nbegal(T1,T2)` retourne 1.
- 9) Déterminer la complexité de cette fonction (le nombre de comparaisons) en fonction de la taille n des tableaux.

Cette fois on cherche à déterminer les valeurs communes dans deux tableaux, mais sans demander une position commune. Pour simplifier les cas particuliers, on supposera que chaque tableau est constitué de valeurs distinctes, mais les deux tableaux n'ont pas nécessairement la même taille.

Par exemple si $T_1 = [1, 3, 8, 7, 5, 0]$ et $T_2 = [7, 3, 8, 2]$, il y a trois valeurs communes.

- 10) Écrire une fonction `nbglobalegal(tab1, tab2)` qui fait le travail demandé.
- 11) Déterminer la complexité de cette fonction (le nombre de comparaisons) en fonction des tailles n et p des tableaux.

Partie C - Recherches dans un tableau trié

On considère la fonction suivante, où le tableau d'entiers T est trié, et x est un entier :

```
1 def recherche(x, T):
2     val = T[0]
3     for k in range(len(T)):
4         if x == val :
5             return True
6         else :
7             val = val + 1
8     return False
```

Cette fonction est censé renvoyer `True` ou `False` selon que x soit dans T ou non, mais est mal codée.

- 12) Proposer un tableau T et une valeur x tels que `recherche(x, T)` donne `True` alors qu'il devrait donner `False`
- 13) Proposer un tableau T et une valeur x tels que `recherche(x, T)` donne `False` alors qu'il devrait donner `True`
- 14) Écrire une fonction dichotomique qui effectue cette recherche par dichotomie
- 15) Rappeler la complexité de la méthode dichotomique par rapport à la recherche séquentielle.

EXERCICE 3 - ETUDE DE HOULE

On s'intéresse à des mesures de niveau de la surface libre de la mer effectuées par une bouée. Cette bouée contient un ensemble de capteurs incluant un accéléromètre vertical qui fournit, après un traitement approprié, des mesures à étudier. Les mesures réalisées à bord de la bouée sont envoyées par liaison radio à une station à terre où elles sont enregistrées, contrôlées et diffusées pour servir à des études scientifiques. Pour plus de sûreté, les mesures sont aussi enregistrées sur une carte mémoire interne à la bouée.

Analyse des mesures

On considère ici que la mesure de houle est représentée par un signal $\eta(t) \in \mathbb{R}$, $t \in [0, T]$ avec η une fonction C^1 . On appelle niveau moyen m la moyenne de $\eta(t)$ sur $[0, T]$. On définit $Z_1, Z_2, \dots, Z_n$ l'ensemble (supposé fini) des Passages par le Niveau moyen en Descente (PND) (voir la figure). A chaque PND, le signal traverse la valeur m en descente. On suppose $\eta(0) > m$ et $\frac{d\eta}{dt}(0) > 0$. On en déduit que $\eta(t) - m \geq 0$ sur $[0, Z_1]$.

Les hauteurs de vagues H_i sont définies par les différences :

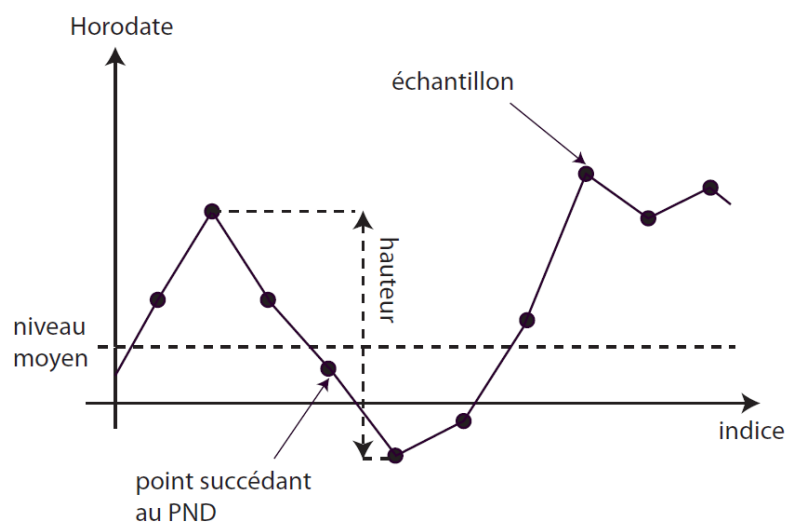
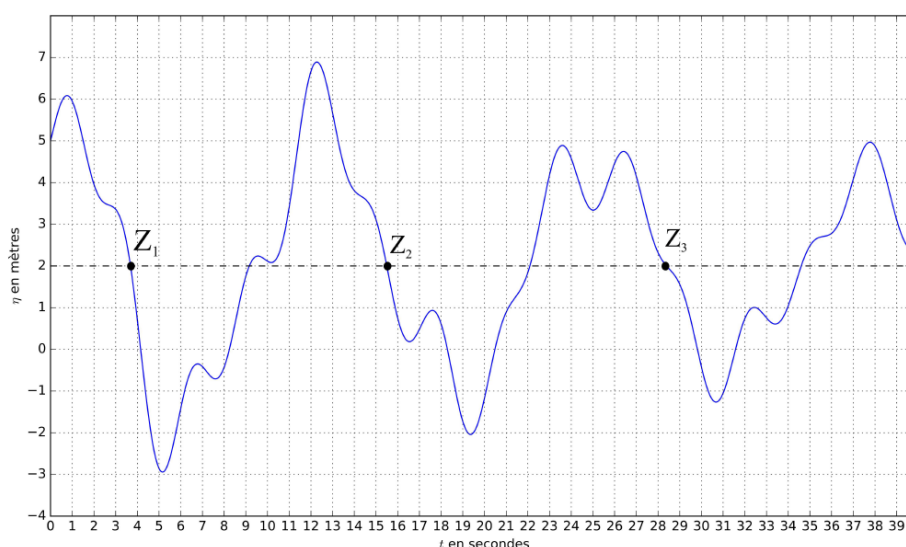
$$\begin{cases} H_1 = \max_{t \in [0, Z_1]} \eta(t) - \min_{t \in [Z_1, Z_2]} \eta(t) \\ H_i = \max_{t \in [Z_{i-1}, Z_i]} \eta(t) - \min_{t \in [Z_i, Z_{i+1}]} \eta(t) \end{cases}$$

On définit les *périodes de vagues* par $T_i = Z_{i+1} - Z_i$.

- 1) Pour le signal représenté sur cette, que valent approximativement H_1 , H_2 et H_3 ? Que valent approximativement T_1 et T_2 ?
- 2) Proposer une fonction moyenne prenant en argument une liste non vide `liste_niveaux`, et retournant sa valeur moyenne.

On souhaite stocker dans une liste `successeurs` les indices des points succédant (strictement) aux PND.

- 3) On propose la fonction `construction_successeurs` qui retourner la liste des successeurs. Recopier et compléter les lignes 6 et 7 de cette fonction.



```

1 def construction_successeurs(liste_niveaux):
2     n = len(liste_niveaux)
3     successeurs = []
4     m = moyenne(liste_niveaux)
5     for i in range(n-1):
6         if # A compléter
7           # A compléter
8     return successeurs

```

- 4) Proposer une fonction `decompose_vagues(liste_niveaux)` qui permet de décomposer une liste de niveaux en liste de vagues. On omettra les données précédant le premier PND et celles succédant au dernier PND. Ainsi `decompose_vagues([1, -1, -2, 2, -2, -1, 6, 4, -2, -5])` (noter que cette liste est de moyenne nulle) retournera `[[-1, -2, 2], [-2, -1, 6, 4]]`

On désire maintenant caractériser les vagues. Ainsi, on cherche à concevoir une fonction `proprietes(liste_niveaux)` retournant une liste de listes à deux éléments $[H_i, T_i]$ permettant de caractériser *chacune des vagues* i par ses attributs :

- H_i , sa hauteur en mètres (m)
 - T_i , sa période en secondes (s).
- 5) Proposer une fonction `proprietes(liste_niveaux)` réalisant cet objectif. On pourra utiliser les fonctions de Python `max(L)` et `min(L)` qui retournent le maximum et le minimum d'une liste L , respectivement.